

01 — Storage Architecture

Module: PostgreSQL Internals | **Difficulty:** Advanced | **Est. reading time:** 45 min

Table of Contents

1. [Learning Objectives](#)
 2. [Overview](#)
 3. [The Data Directory \(\\$PGDATA\)](#)
 4. [Tablespaces](#)
 5. [The base/ Directory](#)
 6. [The global/ Directory](#)
 7. [The pg_wal/ Directory](#)
 8. [File Forks](#)
 9. [OID-Based Filenames](#)
 10. [Segment Files and the 1 GB Limit](#)
 11. [ASCII Diagram: \\$PGDATA Layout](#)
 12. [Live Observation Queries](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions](#)
 17. [Exercises with Solutions](#)
 18. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Describe the complete layout of `$PGDATA` and explain the purpose of every top-level directory.
 - Explain how tablespaces map to filesystem paths and why they matter for I/O isolation.
 - Identify a relation's physical files from nothing but its OID and understand the segment-file naming convention.
 - Distinguish the four file forks (main, fsm, vm, init) and state what each contains.
 - Predict the impact of moving a tablespace or changing `FILLFACTOR` on physical storage layout.
 - Use system catalog queries to locate the physical file for any table or index without leaving `psql`.
-

Overview

PostgreSQL stores all data as ordinary files in the operating system filesystem. There is no special block device, no proprietary container format. This design makes backups, inspection, and disaster recovery straightforward — at the cost of requiring a clear internal convention for how files are named, organised, and extended.

Understanding storage architecture is the entry point for every other internals topic:

- **WAL** writes to `pg_wal/`.
- **Heap pages** live in files under `base/`.
- **VACUUM** rewrites free-space maps (`.fsm` forks) and visibility maps (`.vm` forks).

- **TOAST** tables occupy their own OID-derived files, also under `base/`.

The Data Directory (\$PGDATA)

`$PGDATA` is the environment variable (and the value of `data_directory` GUC) that points to the cluster's root directory. Everything PostgreSQL needs to run lives here unless you explicitly relocate items (WAL to a separate mount, tablespaces to symlinks, etc.).

```
$PGDATA/
├─ PG_VERSION                # single-line file, e.g. "16"
├─ postgresql.conf           # main configuration
├─ postgresql.auto.conf      # overrides written by ALTER SYSTEM
├─ pg_hba.conf               # host-based authentication rules
├─ pg_ident.conf             # user-name maps
├─ postmaster.pid            # PID + port + socket path (runtime)
├─ postmaster.opts           # command-line used to start server
├─ base/                     # one sub-directory per database OID
├─ global/                   # cluster-wide tables (pg_database, pg_roles ...)
├─ pg_wal/                   # write-ahead log segments (16 MB each by default)
├─ pg_xact/                  # commit-status bits for transactions
├─ pg_multixact/             # multi-transaction status (row-level locking)
├─ pg_subtrans/              # subtransaction parent info
├─ pg_twophase/              # prepared (2PC) transaction state
├─ pg_stat/                  # permanent stats collected by stats collector
├─ pg_stat_tmp/              # ephemeral stats files (in-memory on modern PG)
├─ pg_tblspc/                # symlinks → user-defined tablespace directories
├─ pg_replslot/              # logical/physical replication slot state
├─ pg_logical/               # logical decoding state
├─ pg_snapshots/             # exported transaction snapshots
├─ pg_serial/                # serialisable transaction data
└─ pg_notify/                # LISTEN/NOTIFY queue state
```

Key files

File	Purpose
PG_VERSION	Prevents an incompatible binary from opening the directory.
postgresql.conf	Human-editable parameters. Many need SIGHUP or restart to take effect.
postgresql.auto.conf	Written by ALTER SYSTEM. Overrides postgresql.conf at startup.
postmaster.pid	Created at startup, deleted at clean shutdown. Stale file = crash.

Tablespaces

A **tablespace** is a named storage location — a directory path — that you can use when creating databases, tables, or indexes. By default PostgreSQL ships with two tablespaces:

Tablespace	OID	Location
------------	-----	----------

pg_default	1663	\$PGDATA/base/
pg_global	1664	\$PGDATA/global/

User-defined tablespaces point to directories outside `$PGDATA`. PostgreSQL creates a symlink in `$PGDATA/pg_tblspc/<oid>` → external directory, and inside that directory writes a `PG_<major>_<catalogue_version>/` versioned subdirectory, then database-OID sub-directories, then relation files.

```
-- Create a tablespace on a fast NVMe mount
CREATE TABLESPACE fast_nvme
  OWNER postgres
  LOCATION '/mnt/nvme/pg_tblspc';

-- Move a table to the fast tablespace
ALTER TABLE orders SET TABLESPACE fast_nvme;

-- Inspect all tablespaces
SELECT oid, spcname, pg_tablespace_location(oid) AS path
FROM pg_tablespace;
```

Why tablespaces matter

1. **I/O isolation** — put hot tables on NVMe, cold archives on spinning rust.
2. **Capacity management** — database objects can span multiple filesystems.
3. **Testing** — create a tablespace on a RAM filesystem (`tmpfs`) for test data.

Warning: Moving a table between tablespaces rewrites the entire relation and holds `ACCESS EXCLUSIVE` lock for the duration.

The base/ Directory

```
$PGDATA/base/
├─ 1/          # template1 database (OID 1)
├─ 4/          # postgres database default
├─ 16384/      # your first user database
│   ├─ 16385      # heap file for some table
│   ├─ 16385_fsm   # free-space map fork
│   ├─ 16385_vm    # visibility map fork
│   └─ 16386      # another relation
│   └─ ...
└─ PG_VERSION
```

Each directory name is the **OID of the database**. Inside, each file (or group of files) represents one relation fork. The primary filename is the relation's **filenode** number (which usually equals the OID, but can differ after `VACUUM FULL`, `CLUSTER`, `TRUNCATE`, or `ALTER TABLE ... SET TABLESPACE`).

```
-- Find which database has OID 16384
SELECT datname, oid FROM pg_database WHERE oid = 16384;
```

```
-- Find the file path for a specific table
SELECT pg_relation_filepath('orders');
-- Result: base/16384/16385
```

The global/ Directory

`$PGDATA/global/` stores **cluster-wide** relations — objects that must be visible regardless of which database you are connected to. These include:

Catalog	Purpose
pg_database (OID 1262)	One row per database in the cluster
pg_authid (OID 1260)	Roles (users/groups)
pg_auth_members (OID 1261)	Role membership
pg_tablespace (OID 1213)	Tablespace definitions
pg_shdepend	Cross-database dependencies
pg_shdescription	Comments on shared objects

These files use the tablespace `pg_global` (OID 1664) and their paths are literally `$PGDATA/global/<filenode>` .

The pg_wal/ Directory

WAL (Write-Ahead Log) segments live here. Each segment is exactly **wal_segment_size** bytes (default 16 MB, configurable at `initdb` time). Files are named with a 24-character hexadecimal name:

```
000000010000000000000001
AAAAAAAAAAAAAAAAAAAAAAAA
timeline LSN-derived segment number
```

The directory should ideally reside on a separate, fast, dedicated disk because WAL writes are sequential and latency-sensitive. PostgreSQL can be told to write WAL elsewhere at `initdb` time:

```
initdb -D /var/lib/postgresql/data --waldir=/mnt/wal_disk/pg_wal
```

After which `$PGDATA/pg_wal` becomes a symlink to `/mnt/wal_disk/pg_wal` .

File Forks

Every relation can have up to four **forks** — different logical streams of pages for the same relation, stored as separate files with suffixed names:

Fork	Suffix	Contents	Size
Main	(none)	Actual heap or index data pages	Up to many GB

FSM	<code>_fsm</code>	Free-space map: bytes of free space per page	~1 page per 8168 heap pages
VM	<code>_vm</code>	Visibility map: 2 bits per heap page	~1 page per 32768 heap pages
Init	<code>_init</code>	Unlogged-table init fork: zeroed copy used after crash	Only for UNLOGGED tables

```

16385      ← main fork (heap data)
16385_fsm  ← free space map
16385_vm   ← visibility map
16385_init ← init fork (only if UNLOGGED)

```

The FSM and VM are described in detail in files `09_free_space_map.md` and `08_visibility_map.md`.

OID-Based Filenames

When PostgreSQL creates a relation it assigns two numbers:

- **OID** (`pg_class.oid`) — the object identifier, allocated from a global counter.
- **relfilenode** (`pg_class.relfilenode`) — the actual filename used on disk.

Initially `relfilenode == OID`. They diverge after any operation that **rewrites** the relation file:

- `VACUUM FULL`
- `CLUSTER`
- `TRUNCATE`
- `ALTER TABLE ... SET TABLESPACE`

```

-- Confirm they match (or not)
SELECT relname, oid, relfilenode,
       oid = relfilenode AS still_same
FROM   pg_class
WHERE  relname = 'orders';

-- Full physical path
SELECT pg_relation_filepath('orders');

```

The `pg_filenode.map` file

Certain critical system catalogs (like `pg_class` itself) are needed before the catalog is readable. Their filenode numbers are hardcoded in `$PGDATA/global/pg_filenode.map` and `$PGDATA/base/<dboid>/pg_filenode.map`.

Segment Files and the 1 GB Limit

A single relation file is capped at **1 GB** (the value of `RELSEG_SIZE × BLCKSZ = 131072 × 8192`). When a relation grows past 1 GB, PostgreSQL automatically creates a second segment file:

```
16385          ← segment 0 (bytes 0 - 1,073,741,823)
16385.1        ← segment 1 (bytes 1,073,741,824 - 2,147,483,647)
16385.2        ← segment 2
...
```

The segment number is simply appended with a dot. There is no practical limit on the number of segments, so relations can grow to the filesystem size limit.

Note: The 1 GB segment limit exists for portability — some older filesystems and OS APIs have 32-bit file-offset limits. Modern Linux with `O_LARGEFILE` would not need it, but the limit remains for compatibility.

ASCII Diagram: \$PGDATA Layout

```
$PGDATA/
|
|— postgresql.conf ————— tunable parameters
|— pg_hba.conf ————— authentication rules
|— postmaster.pid ————— runtime lock file
|
|— base/ ————— per-database storage
|   |— 1/          template1
|   |— 4/          postgres (default)
|   |— 16384/      myapp_db
|       |— 16385          ← orders (main fork, segment 0)
|       |— 16385.1      ← orders (main fork, segment 1, >1GB)
|       |— 16385_fsm    ← orders free-space map
|       |— 16385_vm     ← orders visibility map
|       |— ...
|
|— global/ ————— cluster-wide catalogs
|   |— 1260          pg_authid
|   |— 1262          pg_database
|   |— pg_filenode.map
|
|— pg_wal/ ————— write-ahead log
|   |— 000000010000000000000001 (16 MB each)
|   |— 000000010000000000000002
|   |— archive_status/
|
|— pg_xact/ ————— commit-status bits (SLRU)
|— pg_tblspc/ ————— symlinks to tablespaces
|   |— 24601 → /mnt/nvme/pg_tblspc/PG_16_202307071/
|
|— pg_replslot/ ————— replication slot state
```

Live Observation Queries

```
-- 1. Show all databases with their OIDs (maps to base/ subdirectory names)
SELECT oid, datname, pg_size_pretty(pg_database_size(datname)) AS size
```

```

FROM pg_database
ORDER BY pg_database_size(datname) DESC;

-- 2. Find the physical file for any relation
SELECT relname,
       pg_relation_filepath(oid) AS filepath,
       pg_size_pretty(pg_relation_size(oid)) AS main_fork_size,
       pg_size_pretty(pg_total_relation_size(oid)) AS total_size
FROM pg_class
WHERE relkind = 'r' -- ordinary tables only
      AND relnamespace = (SELECT oid FROM pg_namespace WHERE nsname = 'public')
ORDER BY pg_total_relation_size(oid) DESC
LIMIT 20;

-- 3. Check if OID and relfilenode have diverged (indicates rewrite happened)
SELECT relname, oid, relfilenode,
       CASE WHEN oid <> relfilenode THEN 'REWRITTEN' ELSE 'original' END AS status
FROM pg_class
WHERE relkind IN ('r','i')
      AND relnamespace = (SELECT oid FROM pg_namespace WHERE nsname = 'public');

-- 4. List all tablespaces
SELECT t.oid,
       t.spcname,
       pg_tablespace_location(t.oid) AS location,
       pg_size_pretty(pg_tablespace_size(t.oid)) AS size,
       r.rolname AS owner
FROM pg_tablespace t
JOIN pg_roles r ON r.oid = t.spcowner;

-- 5. Find relations in a non-default tablespace
SELECT n.nspname, c.relname, c.relkind, t.spcname
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
JOIN pg_tablespace t ON t.oid = c.reltablespace
WHERE c.reltablespace <> 0 -- 0 means "use database default"
ORDER BY t.spcname, n.nspname, c.relname;

-- 6. Show WAL configuration
SHOW wal_segment_size; -- default 16777216 (bytes)
SHOW data_directory;

-- 7. Count WAL files currently on disk (requires superuser)
SELECT count(*) AS wal_files,
       pg_size_pretty(count(*) * 16 * 1024 * 1024) AS approx_wal_size
FROM pg_ls_waldir();

-- 8. Show fork sizes separately
SELECT relname,
       pg_size_pretty(pg_relation_size(oid, 'main')) AS main,
       pg_size_pretty(pg_relation_size(oid, 'fsm')) AS fsm,
       pg_size_pretty(pg_relation_size(oid, 'vm')) AS vm,

```

```

        pg_size_pretty(pg_relation_size(oid, 'init'))      AS init
FROM    pg_class
WHERE   relkind = 'r'
        AND relnamespace = (SELECT oid FROM pg_namespace WHERE nsname = 'public')
ORDER BY pg_relation_size(oid, 'main') DESC
LIMIT  15;

-- 9. Show segment files for a large table (requires pg_ls_dir privilege)
SELECT pg_ls_dir('base/' ||
               (SELECT oid FROM pg_database WHERE datname = current_database())::text)
AS filename
ORDER BY 1;

```

Common Mistakes

1. **Editing files in `$PGDATA` by hand.** Even opening a relation file with a hex editor while the server is running can corrupt it. Always use PostgreSQL APIs.
2. **Confusing OID with `relfilenode`.** After `VACUUM FULL` the OID stays the same but the file on disk has a new name (the new `relfilenode`). Scripts that hardcode the OID as a filename will break.
3. **Assuming the `global/` directory is just another database.** It does not have a database OID subdirectory; its files are accessed directly from `$PGDATA/global/`.
4. **Forgetting about segment files.** Monitoring scripts that check a single file per relation will miss the `.1`, `.2`, ... segments and under-report table size.
5. **Moving tablespaces without `pg_dump / ALTER TABLE ... SET TABLESPACE`.** Physically moving files and recreating the symlink is unsupported and will corrupt the cluster.
6. **Ignoring the `_init` fork for UNLOGGED tables.** After a crash, PostgreSQL truncates the main and fsm/vm forks and copies the `_init` fork back. If you manually inspect the table right after a crash you will see it empty — that is by design.

Best Practices

- **Separate mounts** — put `$PGDATA`, `pg_wal/`, and user tablespaces on separate physical devices or LUNs.
- **Monitor tablespace disk usage** using `pg_tablespace_size()` in your monitoring system.
- **Use `pg_relation_filepath()`** rather than hardcoding paths in tooling.
- **Set `wal_dir` at `initdb` time** if you want WAL on a dedicated disk — changing it later requires a symlink dance.
- **Pre-create tablespace directories** with correct ownership (`postgres:postgres`, mode `0700`) before `CREATE TABLESPACE`.
- **Avoid `VACUUM FULL`** in production during business hours — it rewrites the entire file and holds `ACCESS EXCLUSIVE` lock.
- **Track `relfilenode` resets** in change-management processes — backup scripts that rely on filenames need to be re-validated after any table rewrite.

Performance Considerations

Scenario	Storage Impact	Recommendation
Large table on slow disk	High I/O latency for every scan	Move to a faster tablespace with ALTER TABLE
WAL and data on same disk	WAL writes compete with data reads	Dedicate a separate fast disk to pg_wal/
Many small tables	Thousands of tiny files; directory stat overhead	Batch tiny data into larger tables or partitions
Table just above 1 GB	Segment split is transparent	No action needed; just be aware for monitoring
UNLOGGED tables	No WAL overhead; data lost on crash	Use for truly ephemeral/recreatable data only
Tablespace on tmpfs	Extremely fast; data lost on reboot	Test environments only

Interview Questions

Q1. What is `$PGDATA` and what does it contain?

The root directory of a PostgreSQL cluster. It contains configuration files (`postgresql.conf` , `pg_hba.conf`), the `base/` directory (per-database heap files), `global/` (cluster-wide catalogs), `pg_wal/` (WAL segments), transaction status directories (`pg_xact/` , `pg_multixact/`), and runtime files (`postmaster.pid`).

Q2. What is the difference between a table's OID and its relfilenode?

The OID is the permanent logical identifier stored in `pg_class.oid` . The relfilenode is the actual filename on disk. Initially they are equal. After any operation that rewrites the relation file (`VACUUM FULL` , `CLUSTER` , `TRUNCATE` , `ALTER TABLE ... SET TABLESPACE`), the relfilenode changes but the OID stays the same.

Q3. Why does PostgreSQL limit segment files to 1 GB?

For portability across operating systems and filesystems that may not support files larger than 2 GB (32-bit `off_t`). The limit is $RELSEG_SIZE \times BLCKSZ = 131072 \times 8192 \text{ bytes} = 1 \text{ GiB}$.

Q4. What are the four file forks and what does each contain?

Main (heap/index data), FSM (free-space map), VM (visibility map, 2 bits per heap page), and Init (zeroed copy for UNLOGGED tables used after crash recovery).

Q5. How would you find the physical path of a table named `orders` without leaving `psql` ?

```
SELECT pg_relation_filepath('orders');
```

Q6. Why is the `global/` directory special?

It stores cluster-wide catalogs (`pg_database` , `pg_authid` , `pg_tablespace`) that must be accessible before any specific database connection is made. It uses the special tablespace OID 1664 (`pg_global`) and lives directly under `$PGDATA/global/` .

Q7. What happens to `$PGDATA/pg_wal` if you run `initdb --waldir=/mnt/wal` ?

PostgreSQL creates the WAL files in `/mnt/wal` and places a symlink at `$PGDATA/pg_wal` → `/mnt/wal` . The cluster behaves identically from the application perspective but WAL I/O goes to the separate mount.

Q8. You notice a table's OID is 16385 but `pg_class.relfilenode` is 24601. What likely happened?

The table was rewritten at some point — most likely by `VACUUM FULL` , `CLUSTER` , or `ALTER TABLE ... SET TABLESPACE` . The logical identity (OID 16385) is preserved in the catalog, but the physical file is now named `24601` .

Q9. How do tablespace symlinks work internally?

`CREATE TABLESPACE` creates a directory symlink in `$PGDATA/pg_tblspc/<oid>` pointing to the user-supplied path. Inside that path PostgreSQL creates a version directory `PG_<major>_<catver>/` and then database-OID subdirectories, just like `base/` .

Q10. Can two databases in the same cluster share the same OID directory under `base/` ?

No. Each database gets a unique OID and a unique subdirectory. The directory name is always the OID, which is globally unique within the cluster.

Exercises with Solutions

Exercise 1 — Map a table to its physical file

Task: For the `pg_class` catalog table, determine its OID, relfilenode, tablespace, and the full filesystem path.

```
-- Solution
SELECT c.relname,
       c.oid,
       c.relfilenode,
       t.spcname           AS tablespace,
       pg_relation_filepath(c.oid) AS filepath
FROM   pg_class c
LEFT   JOIN pg_tablespace t ON t.oid = c.reltablespace
WHERE  c.relname = 'pg_class';
```

Expected: `tablespace` will be empty (meaning `pg_default`), `filepath` will be `base/<dboid>/pg_class_filenode` .

Exercise 2 — Count segment files

Task: Write a query to find all tables with more than one segment file (i.e., larger than 1 GB).

```
-- Solution: tables with main fork > 1 GB
SELECT relname,
       pg_size_pretty(pg_relation_size(oid)) AS main_fork_size,
       ceil(pg_relation_size(oid)::numeric / (1024^3)) AS segments
FROM   pg_class
WHERE  relkind = 'r'
```

```
AND pg_relation_size(oid) > 1073741824
ORDER BY pg_relation_size(oid) DESC;
```

Exercise 3 — Tablespace inventory

Task: List every user-created tablespace, its location, owner, and total size.

```
-- Solution
SELECT t.spcname,
       pg_tablespace_location(t.oid) AS path,
       r.rolname AS owner,
       pg_size_pretty(pg_tablespace_size(t.oid)) AS size
FROM   pg_tablespace t
JOIN   pg_roles r ON r.oid = t.spcowner
WHERE  t.spcname NOT IN ('pg_default', 'pg_global')
ORDER BY pg_tablespace_size(t.oid) DESC;
```

Exercise 4 — Detect rewritten tables

Task: Find all tables in the `public` schema whose relfilenode differs from their OID.

```
-- Solution
SELECT relname, oid, relfilenode
FROM   pg_class
WHERE  relkind = 'r'
      AND relnamespace = (SELECT oid FROM pg_namespace WHERE nspname = 'public')
      AND oid <> relfilenode
ORDER BY relname;
```

Cross-References

- **02_heap_storage.md** — Internal structure of the files located in `base/`
- **04_wal.md** — Contents and lifecycle of files in `pg_wal/`
- **05_checkpoints.md** — How dirty pages are flushed during checkpoint
- **08_visibility_map.md** — The `_vm` fork in detail
- **09_free_space_map.md** — The `_fsm` fork in detail
- **10_system_catalogs.md** — `pg_class` , `pg_tablespace` , `pg_database` details